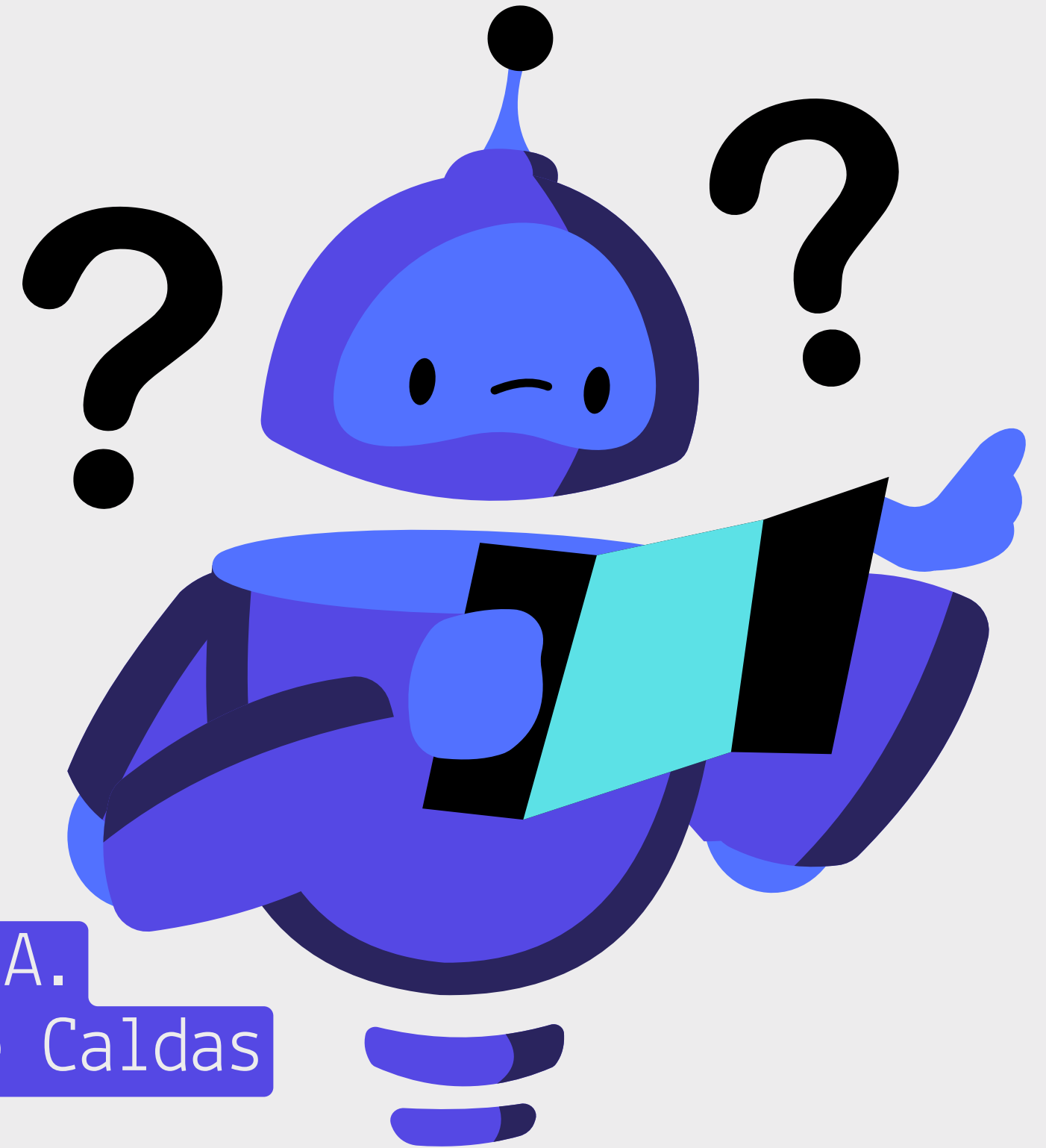


Building a reading platform based on Bookmate

Salazar M. Andrés F., Delgado M. Ichel A.
Universidad Distrital Francisco José de Caldas
Departament of Systems Engineering



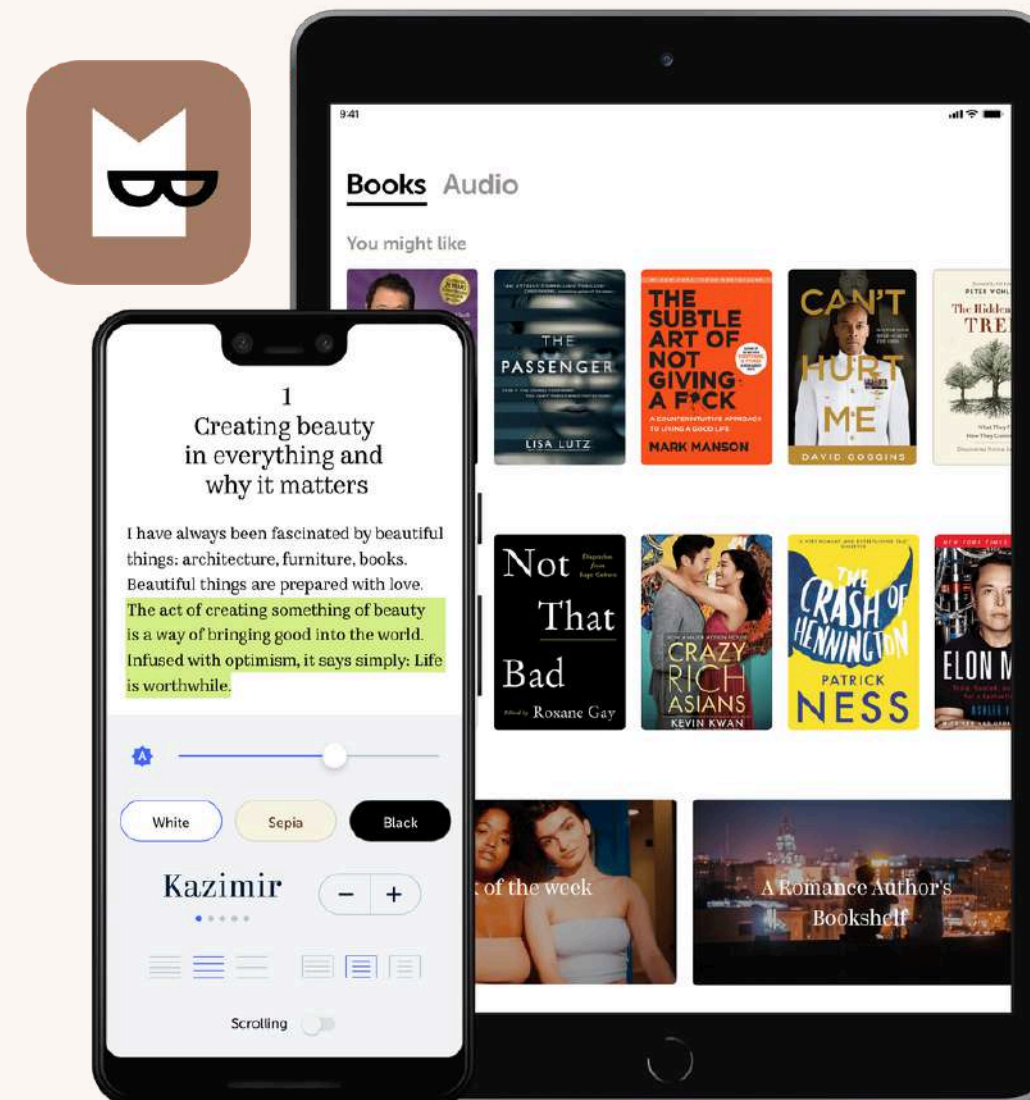
CONTEXTUALIZATION

An all-in-one reading platform based on a monthly subscription model. Users have access to an unlimited catalog of e-books and audiobooks, receive personalized recommendations, and participate in a social network of readers (reviews, book clubs, challenges).



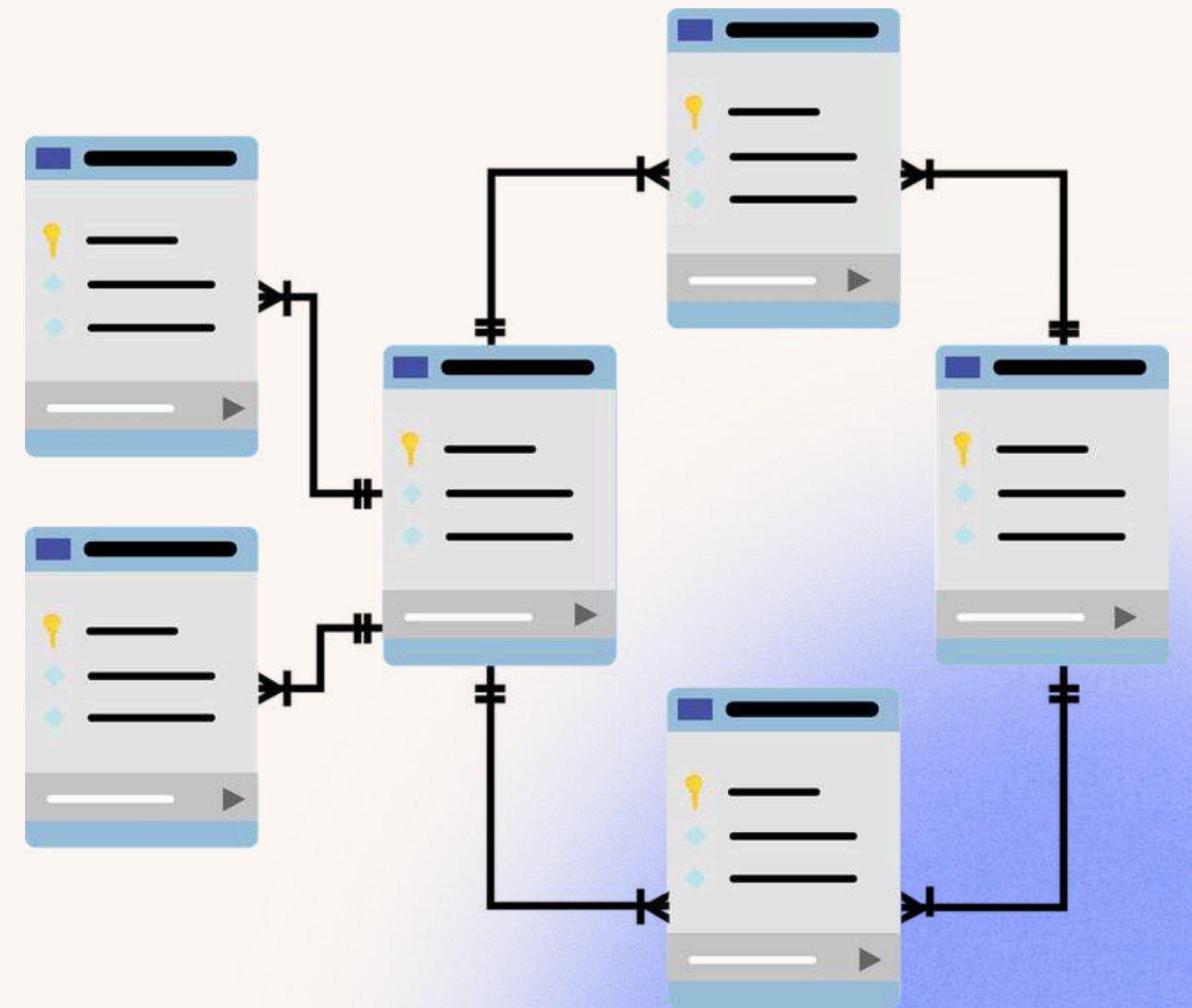
CONTEXTUALIZATION

This project presents the design and implementation of the database architecture for a platform inspired by Bookmate.

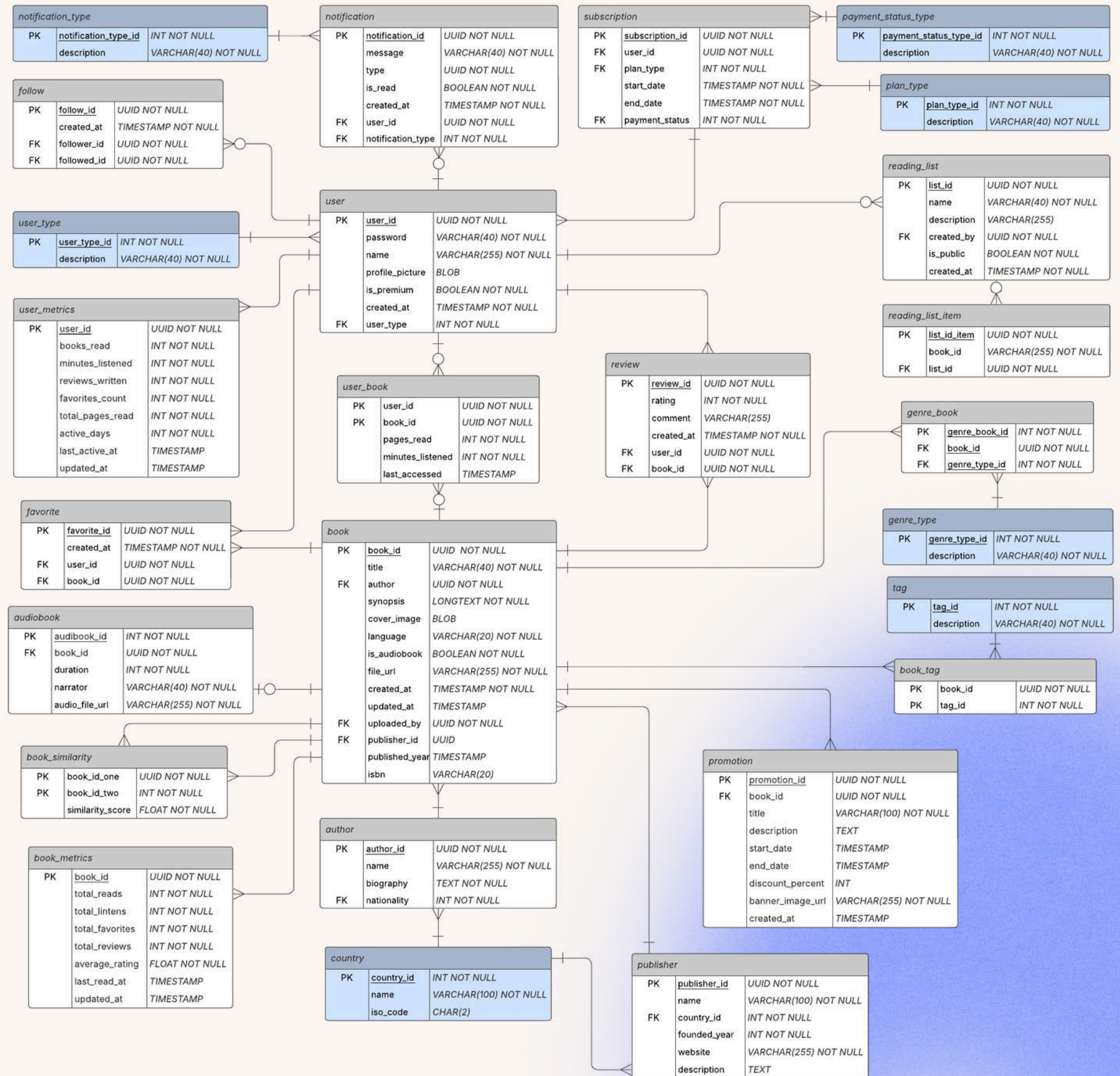
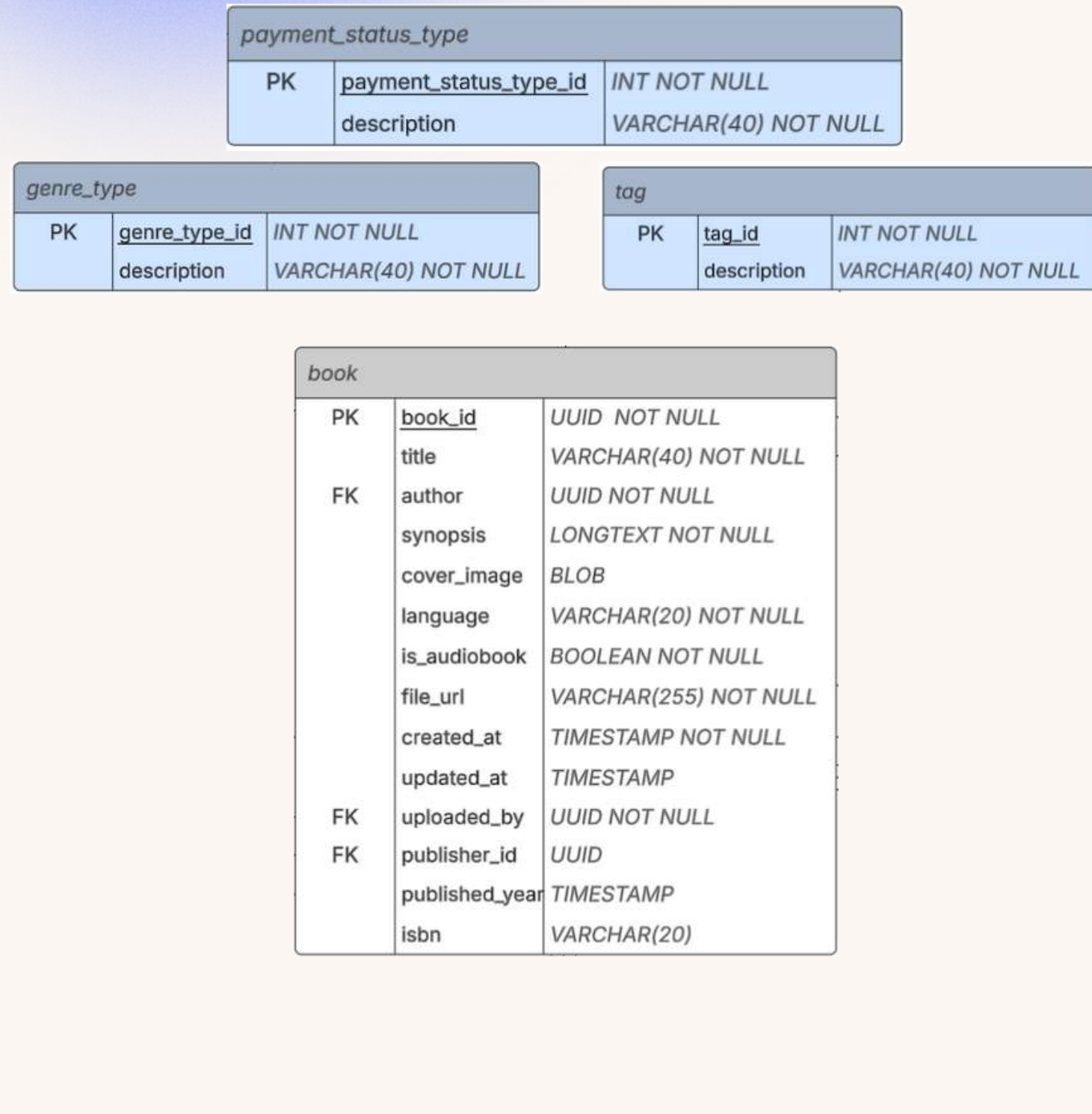


GOAL

This project aims to design and implement a normalized relational database model for a social reading platform. The expected outcome is a scalable schema that supports structured and unstructured content, social interaction, and personalized recommendations.



DATA MODEL DESIGN



TECHNICAL DECISIONS

- user book entity → centralizes user-book interactions (progress, listening time, history), eliminating logic duplication across other tables.
- is active attribute → Content moderation uses logical deletion, preserving data integrity while removing inappropriate content.
- book similarity → stores precomputed similarity scores between books, enabling efficient personalized suggestions.
- book content storage → content files (PDFs, audio) are stored in MongoDB, separating them from relational metadata in PostgreSQL for optimal performance with unstructured data.



TOOL AND TECHNOLOGIES

- Relational model → Lucidchart with crow's foot notation.
- PostgreSQL version 15 → database management system.
DBeaver and terminal → Queries and testing.
- MongoDB version 8 → unstructured content, such as book files and audio tracks.

Books are stored in MongoDB using a flexible JSON structure that supports multiple formats, such as PDF and MP3.

Data preparation & insertion

- Structured data → Python & Faker
- Unstructured data → Python, ReportLab & JSON files.

```
{  
  "id": "UUID",  
  "name": "Book title",  
  "formats": {  
    "pdf": {  
      "fileName": "...",  
      "content": "...",  
      "format": "pdf"  
    },  
    "audio": {  
      "fileName": "...",  
      "content": "...",  
      "format": "mp3"  
    }  
  }  
}
```


RESULTS

TABLE CREATION

```
-- Catalogs/types
CREATE TABLE "user_type" (
  "user_type_id" SERIAL PRIMARY KEY NOT NULL,
  "description" VARCHAR(40) NOT NULL
);

CREATE TABLE "plan_type" (
  "plan_type_id" SERIAL PRIMARY KEY NOT NULL,
  "description" VARCHAR(40) NOT NULL
);

CREATE TABLE "payment_status_type" (
  "payment_status_type_id" SERIAL PRIMARY KEY NOT NULL,
  "description" VARCHAR(40) NOT NULL
);

CREATE TABLE "notification_type" (
  "notification_type_id" SERIAL PRIMARY KEY NOT NULL,
  "description" VARCHAR(40) NOT NULL
);

CREATE TABLE "genre_type" (
  "genre_type_id" SERIAL PRIMARY KEY NOT NULL,
  "description" VARCHAR(40) NOT NULL
);

CREATE TABLE "country" (
  "country_id" SERIAL PRIMARY KEY,
  "name" VARCHAR(100) NOT NULL,
  "iso_code" CHAR(2)
);

CREATE TABLE "tag" (
  "tag_id" SERIAL PRIMARY KEY,
  "description" VARCHAR(40) NOT NULL
);

-- Main tables
CREATE TABLE "user" (
  "user_id" UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  "password" VARCHAR(40) NOT NULL,
  "name" VARCHAR(255) NOT NULL,
  "profile_picture" BYTEA,
  "is_premium" BOOLEAN NOT NULL,
  "created_at" TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

DATA INSERTION

```
-- TEMPORAL TABLE FOR ENTITIES
CREATE TEMP TABLE uuid_temp_entities AS
SELECT
  md5(random()::text || clock_timestamp()::text || row_number() OVER ()):uuid AS id_user,
  md5(random()::text || clock_timestamp()::text || row_number() OVER ()):uuid AS id_book,
  md5(random()::text || clock_timestamp()::text || row_number() OVER ()):uuid AS id_author,
  md5(random()::text || clock_timestamp()::text || row_number() OVER ()):uuid AS id_publisher,
  md5(random()::text || clock_timestamp()::text || row_number() OVER ()):uuid AS id_review,
  md5(random()::text || clock_timestamp()::text || row_number() OVER ()):uuid AS id_favorite,
  md5(random()::text || clock_timestamp()::text || row_number() OVER ()):uuid AS id_follow,
  md5(random()::text || clock_timestamp()::text || row_number() OVER ()):uuid AS id_reading_list,
  md5(random()::text || clock_timestamp()::text || row_number() OVER ()):uuid AS id_subscription,
  md5(random()::text || clock_timestamp()::text || row_number() OVER ()):uuid AS id_notification,
  md5(random()::text || clock_timestamp()::text || row_number() OVER ()):uuid AS id_promotion,
  md5(random()::text || clock_timestamp()::text || row_number() OVER ()):uuid AS id_ad_campaign,
  row_number() OVER ( ) AS rn
FROM generate_series(1, 100000);

-- ===== CATALOGS/TYPES =====
INSERT INTO "user_type" ("user_type_id", "description") VALUES
  (1, 'Administrador'), (2, 'Usuario Premium'), (3, 'Usuario Regular');
INSERT INTO "plan_type" ("description") VALUES
  ('Básico'), ('Premium'), ('Premium Plus');

INSERT INTO "payment_status_type" ("description") VALUES
  ('Pendiente'), ('Pagado'), ('Cancelado'), ('Rechazado');

INSERT INTO "notification_type" ("description") VALUES
  ('Nuevo libro'), ('Recordatorio'), ('Promoción'), ('Seguimiento');

INSERT INTO "genre_type" ("description") VALUES
  ('Ficción'), ('No ficción'), ('Misterio'), ('Romance');
```


RESULTS

DBEAVER

```
SELECT EXISTS (
  SELECT 1 FROM subscription s
  WHERE s.user_id = '6c6d4873-3d45-58e3-4d03-c5f572a16df1'
  AND s.end_date > NOW()
  AND s.payment_status = 2
) AS is_premium_user;
```

The screenshot shows the DBeaver SQL editor interface. At the top, there's a toolbar with icons for grid, SQL, and other functions. Below the toolbar, a SQL query is entered: `SELECT b.book_id, b.title, a.name AS autl`. The query results are displayed in a table with the following columns: `book_id`, `title`, `author name`, `synopsis`, and `language`. The table contains three rows of data. The status bar at the bottom indicates '20 row(s) fetched - 1,240s, on 2025-07-08 at 12:04:14'.

	book_id	title	author name	synopsis	language
1	fdf20a8f-6c94-810f-c97c-236cae848474	Adaptive 24/7 ability	Isabela Rosales	Dignissimos ullam itaque officiis optio in. Architecto	en
2	17df5652-4269-2385-edd2-045852bc7448	Adaptive 24/7 access	Isabela Rosales	Omnis sed velit praesentium quo ex. Modi et tempor it	it
3	c513b42f-8a67-5b07-23d2-0ea61a3d25dc	Adaptive 24/7 array	Isabela Rosales	Atque tenetur labore ab illo. Inventore veniam provi en	en

MONGOSH CONSOLE

```
bookmate> db.book_content.find({'id': 'c0d12e92-70e7-04c8-c758-93f9c0567e06'})
[
  {
    _id: ObjectId('686dd390dba843fa3974f6ee'),
    id: 'c0d12e92-70e7-04c8-c758-93f9c0567e06',
    name: 'Configurable radical solution',
    formats: {
      pdf: {
        fileName: 'Configurable_radical_solution.pdf',
        content: 'JVBERi0xLjMKJZOMi54gUmVwb3J0TGF1IEdlbmVYXR1ZCBQREYgZG9jdW11bnQgaHR0cDovL3A=
VuY29kaW5nIC90YW11IC9GMSAvU3VidHlwZSAvVHlwZTEgLS1R5cGUGL0Zvb3R5b2J0cCjMgMCAvYmoKPdWKL0
V4dCAvSW1hZ2V2IC9JbWFnZUMGL0ltYWdlSSBdCj4+IC9Sb3RhdGUGMCAvVHJhbnMgPDwKCj4+IAogIC9UeXB1IC9QYW
AoYWSvbnltb3VzKSAvQ3J1YXRpb25EYXR1IC9E0jIwMjUwNzA4MjExNzQzLTAlJzAwJykgL0NyZWFi0b3IgKFJlcG9ydE
9ydExhYiBQREYgTG1icmFyeSA0IHd3dy5yZXBvcnRsYW1uY29tKSAKICAvU3ViamVjdCAodW5zcGVjaWZpZWQpIC9UaX
03IDAga2JqCjw8Ci9GaWx0ZXIgaWYAvQVNDUk4NUR1Y29kZSAvRmxdGVEZWNvZGUGXSAvTG9VZ3R0IDE4NAo+PgpzdH
dRWF49bC9TZjo0MT89Rmh1LnAsb1hBR1l0PVt1VjQ0UVVBI1pPPkpnUj8pRnQ7XE9TOTZEWUtAP1hoaU0qaktTZj0tJV
AwMDAga2IAKMDAwMDAwMDIxMSAwMDAwMCAvU3ViamVjdCAodW5zcGVjaWZpZWQpIC9UaX03IDAga2JqCjw8Ci9GaWx0
Zm9udG91bWZlc30TbmNTRhMzlmZTJkY2YxOT5dC1UgUmVwb3J0TGF1IEdlbmVYXR1ZCBQREYgZG9jdW11bnQgLS0gZGlnZX
format: 'pdf'
      }
    }
  }
]
```


RESULTS

DBEAVER

```
-- As a user, I want to be able to see the favorite books and audiobooks of the users I follow to discover new titles.

SELECT b.title,
       b.cover_image,
       b.is_audiobook,
       u."name" AS followed_user_name
FROM follow f
JOIN favorite fav ON fav.user_id = f.followed_id
JOIN book b ON b.book_id = fav.book_id
JOIN "user" u ON u.user_id = f.followed_id
WHERE f.follower_id = '4dc534e7-2b3b-d7cf-ac42-8580921cca8c';
```

SQL Editor: `SELECT b.title, b.cover_image, b.is_audiobook, u.name AS followed_user_name` Enter a SQL expression to filter results (use Ctrl+Space)

	AZ title	cover image	is audiobook	AZ followed user name
1	Secured tertiary website	[NULL]	[]	Máxima María Fernanda Jáuregui España
2	Persistent needs-based methodology	[NULL]	[]	Chelo Pedrero Espinosa
3	Centralized client-driven orchestration	[NULL]	[]	Rosalina Villaverde-Beltrán
4	Synergistic incremental intranet	[NULL]	[]	Flor Flores Roman
5	Monitored intangible infrastructure	[NULL]	[]	Artemio Correa Salas
6	Up-sized global contingency	[NULL]	[]	Jose Francisco Tristán Escrivá Prada
7	Modularized omnichannel infrastructure	[NULL]	[]	Rebeca de la Cruz Delgado

29 row(s) fetched - 0.015s (0.001s fetch), on 2025-07-09 at 23:15:54

RESULTS

DBEAVER

```
-- As a user, I want to receive recommendations of books and audiobooks based on my readings and reviews

SELECT
  b.book_id,
  b.title,
  b.cover_image,
  b.is_audiobook,
  MAX(bs.similarity_score) AS score
FROM user_book ub
JOIN book_similarity bs
  ON bs.book_id_one = ub.book_id
JOIN book b
  ON b.book_id = bs.book_id_two
WHERE ub.user_id = 'a1070a38-fbe4-ac15-518c-dca8501b5f85'
  AND b.is_active = TRUE
  AND b.book_id NOT IN (
    SELECT book_id FROM user_book WHERE user_id = 'a1070a38-fbe4-ac15-518c-dca8501b5f85'
  )
GROUP BY b.book_id, b.title, b.cover_image, b.is_audiobook
ORDER BY score DESC
LIMIT 10 OFFSET 0;
```

SELECT b.book_id, b.title, b.cover_image, b.is_audiobook, MAX(bs.similarity_score) AS score

	book id	title	cover image	is audiobook	score
1	0484ccd6-7bfb-bfd4-8d36-84d8ddd1150a	Advanced client-driven application	[NULL]	[v]	1
2	3e0c0343-b6fe-d299-85c6-ee4a5b2e1726	Robust interactive artificial intelligen	[NULL]	[v]	1
3	994c7f05-a2a3-1d6d-4ea4-394851686a51	Multi-lateral human-resource array	[NULL]	[]	1
4	cf8ba25e-7ecb-7b34-3cf4-96e61e270c47	Compatible cohesive strategy	[NULL]	[]	1
5	433af641-e647-6a67-86c5-aa17e1b228af	Mandatory systemic structure	[NULL]	[v]	1
6	570470b6-39ca-cacb-4c11-55a28ec595cb	Optimized multimedia hub	[NULL]	[]	1
7	1a45f0e-4372-170e-0145-f04b045c024	Optimized multi-state infrastructure	[NULL]	[v]	1

10 row(s) fetched - 0.009s (0.001s fetch), on 2025-07-09 at 23:17:02

CONCLUSIONS

01

Successfully designed and implemented a relational database for a social reading platform based on real user stories, supporting features like user profiles, reading tracking, recommendations, and moderation using PostgreSQL.

02

All queries were manually tested and performed correctly, with recommendation queries delivering low-latency results even on large datasets.

03

The database is ready for backend/frontend integration and supports future extensions like gamification, analytics, and AI-driven features.

BIBLIOGRAPHY

- [1] E. SolutionsHub, "Exploring Business Model Canvas examples". Oct. 20, 2023. Available: <https://solutionshub.epam.com/blog/post/business-model-canvas-examples>
- [2] IBM, "Relational Databases," Ibm.com, Oct. 20, 2021. <https://www.ibm.com/think/topics/relational-databases>
- [3] Lucidchart, "Database Design Structure - Schema Tutorial", 2025. <https://www.lucidchart.com/pages/tutorial/database-design-and-structure>
- [4] Atlassian, "Acceptance Criteria Explained [+ Examples & Tips]," Atlassian, 2024. <https://www.atlassian.com/work-management/project-management/acceptance-criteria>
- [5] Ramez E., "FUNDAMENTALS OF FourthEdition DATABASE SYSTEMS.", 2003. [E-book] Available: https://www.uoitc.edu.iq/images/documents/informatics-institute/Competitive_exam/Database_Systems.pdf